

MOSS: 自主智能体系统中 基于源码级重写的自我进化

Qianshu Cai^{1,*},[△] Yonggang Zhang^{2,*} Xianzhang Jia² Huajiang Zheng²

Wei Xue² Jun Song³ Xinmei Tian^{1,†} Yike Guo^{2,†}

¹University of Science and Technology of China

²Hong Kong Generative AI Research & Development Center

²The Hong Kong University of Science and Technology

³Hong Kong Baptist University

摘要

自主智能体系统在部署后基本保持静态：它们不从用户交互中学习，反复出现的故障会一直持续到下一次由人工驱动的更新发布修复为止。自我进化智能体应运而生，但所有现有方法都将进化局限于可通过文本修改的工件——技能文件、提示配置、记忆模式、工作流图——而完全未触及智能体框架本身。由于路由、钩子排序、状态不变量和调度都存在于代码中，而非任何文本工件中，因此一整类结构性故障从文本层面根本无法触及。我们认为，源码级适配是一种本质上更通用的媒介：它是图灵完备的，严格包含所有可通过文本修改的范围，能够确定性地生效而非依赖基座模型的遵从性，并且不会在长上下文漂移下退化。我们提出了 MOSS，一个在生产级智能体基座上执行源码级自我重写的系统。每次进化都锚定于一批自动整理的、来自生产环境的故障证据，并通过确定性的多阶段流水线进行；代码修改被委托给一个可插拔的外部编码智能体命令行接口，而 MOSS 保留阶段顺序和判定结果。候选版本通过在临时试验工作器中针对候选镜像重放该批次进行验证，然后通过用户同意门控的就地容器替换进行升级，并带有健康探测门控的回滚机制。在 OpenClaw 上，MOSS 在无需人工干预的单次循环中，将四项任务的平均评分器得分从 0.25 提升至 0.61。GitHub：
<https://github.com/hkgai-official/Moss>。

1 引言

应用级自主智能体系统（如 OpenClaw [OpenClaw, 2024]）已经从研究演示成熟为现实世界中的工作者，部署在 Slack、Discord、Web 及其他渠道中，为用户完成复杂的多步骤任务。然而，一旦部署，它们基本保持静态：它们不从实际使用方式中学习，相同的故障模式会在不同用户之间反复出现，直到下一次由人工驱动的迭代发布修复为止。一波自我进化智能体应运而生，使智能体能够在部署后自我进化 [Nous Research, 2024, 2026, Wang et al., 2026, Ma et al., 2026, Liang et al., 2026, Wang et al., 2025]。

这些系统有一层未被触及。如表 1 所示，它们的演化范围仅限于文本可变工件——技能文件、提示配置、记忆模式，以及至多工作流* 同等贡献。† 通讯作者。△ HKGAI 与香港科技大学的远程研究助理。

图；智能体骨架——路由、状态管理、分发、钩子、中介器、会话生命周期——从未被智能体自身修改。这一边界所施加的限制是物理性的：对文本可变工件的编辑只能改变智能体自身的想法（说什么、调用哪个技能、如何将任务分解为子目标）；它们无法改变骨架代为做出的决策。一旦故障源于这一层——消息路由错误、钩子乱序触发、会话状态损坏、并发技能间的原子性缺陷——任何对技能、提示或记忆的更新都无法触及它：缺陷不在提示文本中，重写提示也无法掩盖它。此类故障的比例随骨架复杂度而增加，因此随着智能体系统成熟，这一差距会不断扩大。

表 1：应用级自演化智能体系统的演化范围。

项目	技能	提示	记忆	骨架
Hermes Agent [Nous Research, 2024]	✓	×	✓	×
SkillClaw [Ma et al., 2026]	✓	×	×	×
GenericAgent [Liang et al., 2026]	✓	×	✓	×
EvoAgentX [Wang et al., 2025]	✓	✓	×	×
MOSS (Ours)	✓	✓	✓	✓

本文认为，源代码级适应——智能体对其自身源代码执行自我重写——是一种根本上更通用的媒介，在四个维度上同时优于基于文本可变的进化。它是图灵完备的：由于编程语言是图灵完备的，源代码设计空间构成了一个通用搜索空间，其中每一个文本可变的智能体设计空间——提示、技能、记忆模式、工作流图——都作为严格子集存在 [胡等人, 2025 年]；编辑代码不仅能达到这些子集所能表示的所有配置，还能达到它们无法表示的任意智能体结构。它是每一个文本可变范围的严格超集：提示编辑所能实现的任何效果，等效的代码编辑也能实现，反之则不然。它确定性地生效：路由逻辑、钩子顺序和状态机不变量作为代码运行，其行为不依赖于基础模型是否正确读取新文本并遵守它——文本可变的修复则相反，其有效性随当前基础模型能力而起伏。并且它不会在长上下文漂移下退化：文本可变的修复是提示、技能和记忆条目，智能体必须在每一轮重新读取，随着这些条目在数周的生产使用中积累，模型对任何单一指导的遵循度会稀释；源代码层的编辑被编码为行为，而非需要重新读取的文本，因此不会随着系统老化而退化。

学术研究已表明，源级自改写可在最小支架上实现：SICA [Robeyns 等, 2025]、达尔文哥德尔机 [Zhang 等, 2025] 和 HyperAgents [Zhang 等, 2026] 均展示了一个智能体通过修改自身实现来提升其基准分数；Meta-Harness [Lee 等, 2026] 进一步表明，向编码智能体提议者暴露过去的执行迹比仅凭基准分数更能驱动迭代改进。但这些系统都运行在最小支架上，反馈由基准分数门控——本质上是一种探索性范式。将同样的源级能力迁移到应用级、生产级基座上，则是一个完全不同的工程环境：代码库庞大，没有干净的基准分数来锚定演化，且实时用户流量与持久用户状态必须得以保留。这促使一系列具体问题上升到系统设计层面：何时触发演化尝试，修复应落在代码库的何处，生成的候选版本是否确实优于已部署版本，以及如何在干扰实时部署的情况下将该候选版本呈现给真实用户。

本文提出 MOSS，一个在生产级智能体基座上执行全面源级自改写的系统——这是上述源级适应的一个实例。如表 1 所示，它是唯一达到该框架的系统，覆盖了先前工作所处理的文本可变范围的严格超集。MOSS 将其整个演化生命周期——触发、状态查询、停止、应用、对话标记——通过系统提示注入作为内置的 `moss evo` 命令行能力暴露给智能体基座，因此用户通过他们日常工作中已使用的同一对话接口与演化进行交互。证据从两个互补渠道积累：对最近用户会话的周期性后台扫描自动浮现表现不佳的对话片段，用户还可以在对话中标记任意轮次；两者都汇入同一批次，且每个下游阶段都锚定于修复该批次，而非锚定于合成基准目标。代码修改由可插拔的外部编码智能体命令行工具执行，该工具作为主机侧子进程被调用，候选版本在临时试验工作器上进行验证，这些工作器针对该批次进行回放。

候选镜像运行于生产等效容器中，而非针对单元测试或合成测试框架。在整个循环过程中，所有工件——计划、差异、构建日志、每次迭代的评分矩阵——均可由智能体按需读取，因此用户可在授权替换前审计系统意图变更的内容及原因。收敛时，MOSS 通过同一对话渠道主动通知用户并暂停；仅在用户明确授权后，宿主守护进程才执行容器原位替换，保留用户状态，并在替换后健康探测失败时自动回滚。在 OpenClaw 上，MOSS 在单次循环内将四项任务的平均评分从 0.25 提升至 0.61，全程无需人工干预。

2 系统架构

MOSS 通过五个组件在生产级智能体基座上实现自我重写：运行基座用户面向智能体的主容器；控制面命令行接口（CLI），智能体（以及用户通过智能体）借此驱动演化；可插拔的外部编码智能体 CLI，按阶段调用以进行代码编辑；宿主驻留守护进程（宿主守护进程），监督其余所有组件；以及用于验证候选镜像的临时试验工作器。

2.1 基座

为便于本文阐述与案例研究，MOSS 采用 OpenClaw [OpenClaw, 2024] 作为基座；该设计可适配其他主流智能体基座，如 Hermes Agent。OpenClaw 是一个生产级智能体系统，具备多渠道网关、插件与钩子机制、会话与技能机制以及持久化用户状态。学术自演化智能体所运行的最小脚手架远小于此且更为简单；这一差距塑造了下述四项设计选择。

2.2 控制面

MOSS 通过单一的 moss evo CLI 将整个演化生命周期暴露给智能体基座，基座的用户面向智能体通过其内置 shell 工具调用该 CLI。九个子命令覆盖完整控制面：status、batches、batch、start、stop、restart、apply、flag 和 catch-up。前七个子命令通过 HTTP 路由至 MOSS 注入基座网关的演化控制端点组，并到达容器内演化服务；flag 和 catch-up 通过 Unix 套接字路由至宿主守护进程的自动扫描引擎。

命令行界面（CLI）在安装时通过将脚本单次绑定挂载到容器中，即可从基板运行时内部访问。为使智能体感知该能力，MOSS 在基板的系统提示中注入一段文字，指引智能体查阅磁盘上的能力文档，该文档描述了命令、用法及操作规则；当用户提及演化、批次、应用新版本或对某一轮次表示不满时，智能体会按需读取该文档。从 MOSS 到智能体的异步通知则通过三个网络钩子事件反向传递：演化收敛与演化失败由容器内的演化服务在循环终止时触发，应用完成由宿主守护进程在容器切换稳定后触发。每个事件均被发送到一个网络钩子端点，基板中的钩子映射配置将其转换为系统消息，在智能体的下一轮次中投递。

这一三件套接口——命令行界面（CLI）输入、网络钩子（Webhook）输出、能力文档作为参考——构成了莫斯（MOSS）与底层环境之间的全部契约。任何提供五项原语（等价于外壳的工具执行、文件系统读取、周期性调度、网络钩子到智能体的投递以及系统提示注入）的底层环境，都可以通过配置这些组件来承载莫斯，而无需修改莫斯自身的代码。

2.3 外部编码智能体

在如此庞大的代码基上，MOSS 无法通过单一推理循环读取并重写代码库本身来完成代码修改——跨文件不变量与并发状态交互超出了单次循环所能承载的范围。因此，MOSS 通过主容器内部的确定性状态机处理演化调度与决策，并将具体的编辑操作委托给作为宿主侧子进程调用的外部编码代理命令行界面（CLI）；该 CLI 运行在宿主上而非主容器内，因为只有宿主具备外壳环境与网络。

出口，以及范围挂载编辑工作所需。各阶段的调用机制推迟至第 3.3 节。职责沿此接缝划分：MOSS 负责阶段排序、判定、循环退出及交换时刻，而编码代理命令行界面（CLI）负责在其被赋予的范围内执行编辑操作。

编码智能体 CLI 本身是可插拔的。MOSS 通过一个四方法的运行器接口抽象集成；调度器、RPC 模式、阶段逻辑和基础平台侧网关在不同提供商之间保持不变。树中内置了四个运行器——Claude Code [安思罗匹克公司，2025 年]、OpenAI Codex [开放人工智能公司，2025 年]、DeepSeek-TUI [汉博恩，2025 年] 和 OpenCode [开放代码项目，2025 年]——在启动时通过配置选择，并带有一个可选的每次生成覆盖，允许单个角色在一次进化运行中使用不同的提供商。添加第五个提供商只需一个新的运行器文件加上注册表中的一行。这种解耦使进化机制独立于任何特定的编码智能体供应商，并允许基础平台操作员将 MOSS 与最符合其模型访问和成本约束的提供商配对。

2.4 主机守护进程监督

容器生命周期管理不能驻留在主容器内部：一旦候选方案收敛，容器必须停止并由新构建的镜像替换，而进程在自身退出后无法启动新容器。

MOSS 将这一职责以及若干相关职责置于一个驻留在主机上的异步输入输出进程（称为主机守护进程）中。它提供一个 Unix 套接字远程过程调用，处理四类操作：按阶段调用编码智能体命令行接口（§ 2.3）、试验工作器容器生命周期、Docker 构建与镜像管理，以及自动扫描引擎，该引擎扫描会话 JSONL 以查找表现不佳的对话片段（§ 3.1）。一个交换监督任务在同一事件循环中运行，通过文件轮询进化状态目录，以获取网关应用处理器写入的交换请求；检测到请求后，它针对候选镜像重启基座容器，探测新容器的健康状况，然后要么提交交换，要么回滚到最近已知良好的镜像（探测协议和状态文件布局在 § 3.5 中描述）。交换完成后，守护进程向刚完成交换的网关触发应用完成网络钩子，以便新实例的智能体收到一条系统消息，宣布刚刚发生的事情。

2.5 拓扑结构

前述四个选择具体化为图 1 中的主机侧拓扑结构。

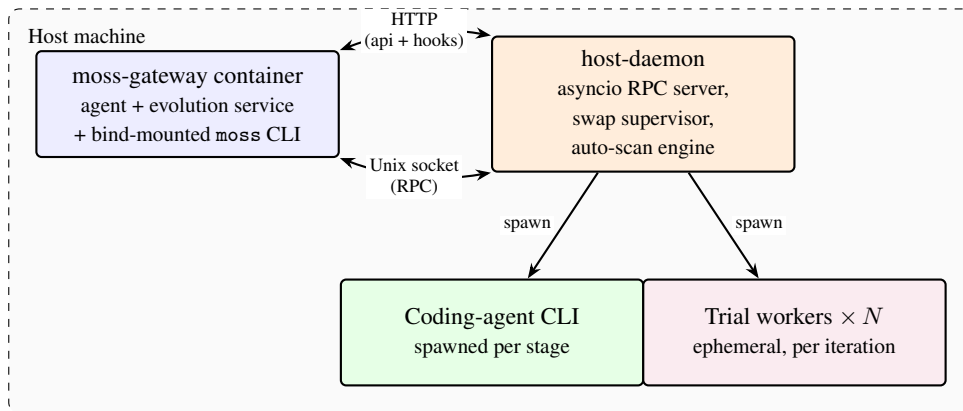


图 1: MOSS 主机侧拓扑结构。moss-gateway 容器承载面向用户的代理、容器内演化服务以及绑定挂载的 moss 命令行界面；主机守护进程（一个异步输入输出进程）为编码代理和试验工作器的编排提供 Unix 套接字远程过程调用服务，通过网关暴露用于演化控制的超文本传输协议路径，监督交换请求，并运行自动扫描引擎。编码代理命令行界面在每个演化阶段被生成；试验工作器在每次迭代时作为临时容器被启动。用户状态卷（会话、内存、凭据、代理配置）从主机文件系统挂载到 moss-gateway 容器中，以便状态在就地交换后得以保留（未显示）。

宿主守护进程（host-daemon）永久运行于宿主机上，并监管另外三个宿主侧实体。苔藓网关（moss-gateway）容器是长期运行、面向用户的基底实例，承载着聊天代理，

容器内进化服务，以及绑定挂载的苔藓命令行界面；其镜像标签在每次成功交换时递增。编码代理命令行界面是守护进程在每个进化阶段（第 3 节）生成的子进程；它仅在该阶段持续期间存在。试验工作器在另一种意义上是短命的：每次迭代从候选镜像（第 3.4 节）启动 N 个容器，自主地针对每个批处理任务运行代理，然后拆除它们。

用户状态——会话、内存、凭证和代理配置——驻留在从主机文件系统挂载到 `moss-gateway` 容器的用户状态卷上。因此，交换会销毁容器但不会销毁卷，新镜像会完整继承状态；试用工作进程在没有此挂载和隔离网络的情况下运行。

3 演化过程

本节完整展示莫斯（MOSS）的一次演化过程：从精选的失败批次到替换入更强智能体——基于真实用户会话证据的定向目标（§ 3.1）、在基线上进行的有界迭代循环（§ 3.2）、每次迭代的七阶段流水线（§ 3.3）、对候选镜像的运行验证（§ 3.4），以及原地容器替换（§ 3.5）。

3.1 定向进化

在学术领域，源代码级的自进化系统遵循探索式范式：每个候选方案根据固定基准进行评分，随后基于这些评分提出随机变异，并保留表现更优的变体以供进一步迭代。这种范式并不适用于生产级智能体系统。代码库规模庞大，随机或无导向的变异很少能产生有用的改动；生产智能体的质量与具体任务相关，不存在基准式的全局适应度信号；在真实用户流量下，任何未经锚定的补丁都无法与回归区分开来；而且用户期望修复的是他们遇到的具体场景，而非对智能体进行全局重新调整。因此，MOSS 采用一种有向且确定性的方法：每次进化都锚定在一批具体的生产故障证据上，并且每个下游阶段都严格绑定于修复该批次。

证据通过两条共享同一后端的命令行接口路径进入批次。一个周期性的定时任务运行 `moss evo catch-up`，该任务从每个会话的游标开始扫描每个智能体的会话 JSONL 文件，并将新内容切分为块；在内联阶段，任务评估阶段对每个块的关键点进行评分，只有被标记为弱或缺失的块才会被追加到该会话当前打开的批次中。与此同时，当用户在对话中表达不满时，智能体会调用 `moss evo flag`，该命令对单个会话执行相同的扫描，从游标到文件末尾，并以相同方式生成块。两条路径都会将内容追加到源会话的打开批次中——每个会话维护一个打开批次，当其块数达到可配置阈值（默认 8）时，该批次会被封存并替换为一个新的打开批次。进化由 `moss evo start` 触发；不带参数时，会选择最新的非空批次，如果该批次仍处于打开状态，则先将其封存。

3.2 进化循环

在此规模下，单次修补并不可靠：定位可能遗漏相关文件，初始方案可能误判根本原因或范围界定有误，首次实现可能使相邻行为发生退化。因此，MOSS 采用迭代方式。每次迭代中对修补候选的结构化评估会为下一次迭代的定位与规划提供依据，从而细化方向，而非从头重试。由于定向进化具有可定义的终止条件——即批次已修复——该循环是有界的：当满足该条件或判定进一步工作无成效时，循环退出。

基线评估在第一次迭代之前运行。任务评估阶段根据选定的关键点集对已附加到每个批次块的原始记录进行评分，生成基线关键点矩阵，为所有后续比较提供锚定（§ 3.4）。

基线评估之后，每次迭代运行七阶段流水线（§ 3.3），并以四种判定之一结束。其中三种为终止性判定：批次已修复（循环退出至 § 3.5）、基础模型已达能力上限、或当前框架内无任何修补可解决该失败。第四种判定——已取得进展但仍需更多工作——则继续进入下一次迭代。停滞保护机制对此提供支持：当连续多次迭代中没有任何关键点

得到改善时，编排器在当前峰值处强制收敛，而非消耗更多试验。

进化深度旋钮——轻量、标准或深度——可调整迭代预算。最大迭代次数、各阶段轮次预算、每任务试验次数以及停滞阈值均同步变化。默认值为标准；面对更严重退化的用户可请求深度模式。

3.3 阶段流水线

一次迭代分解为七个阶段。若要求单个提示同时完成诊断、规划、实现、验证和决策，会导致上下文过载，且输出质量低于按序执行的流程。每个阶段还读取不同的工件——轨迹、诊断、差异、试验输出、跨迭代矩阵——而基于门控的质量控制需要明确的边界：规划与实现之间的审查只有在规划作为独立阶段时才存在。任务评估阶段单独设立还有另一个原因：其输出会被归档并在各次迭代之间进行比较。MOSS 通过在每个阶段调用一次编码智能体命令行接口（§ 2.3）来按序执行这些阶段。构建与试验（§ 3.4）是围绕任务评估阶段的运行时支持，而非推理阶段本身，因此不计入这七个阶段。

七个阶段依次为：

1. 定位——读取基线轨迹与批量失败信息，并撰写诊断报告，不提出修复方案。
2. 规划——识别根本原因并明确修复方案：哪些文件需要修改，添加何种逻辑，哪些保持不变。
3. 规划评审——第一道质量关卡；规划可能被批准、因架构偏离目标而被驳回，或因范围过窄而被驳回。规划与规划评审阶段在轮规划循环中交替进行，直至获得批准或达到轮次预算上限。
4. 实现——在内部基础代码库中编写代码，并作为单个 Git 提交完成。
5. 代码评审——第二道质量关卡；根据规划对差异进行评审，并决定批准或驳回。实现与代码评审阶段在轮代码循环中交替进行，每轮之间工作树会硬重置到该循环的起始提交。
6. 任务评估——在构建与试验（§ 3.4）之后，对每个任务四到七个关键点按四级定性量表进行评分（强 / 合格 / 弱 / 缺失）。同一阶段在循环前的基线（§ 3.2）中运行一次；当时选定的关键点集合在循环剩余部分保持不变。
7. 裁决——将所有逐任务评估结果与跨迭代关键点矩阵综合为四种裁决之一：收敛、NEED_MORE_WORK, FUNDAMENTAL_LIMIT_MODEL, 或 FUNDAMENTAL_LIMIT_ARCHITECTURE。若关键点矩阵在深度相关的平台窗口内未显示改进，编排器可将 NEED_MORE_WORK 降级为收敛。

3.4 运行时验证

代码审查阶段在语法和语义层面运作，因此运行时故障会通过它：竞态条件、跨模块状态交互以及钩子顺序依赖行为仅在代理运行时才会显现。整类应用级故障——通道路由、会话生命周期、跨模块状态传播——只有在执行时才会暴露。因此，验证必须在运行时进行，在等效生产环境中，并针对最初产生故障证据的相同提示。

MOSS 通过临时试验工作器实现这一点。构建之后，宿主守护进程从候选镜像中生成 N 个短生命周期容器——该镜像与收敛时将被提升的镜像相同——并让代理在每个容器内自主处理批量任务，每个任务重复多次试验以暴露不稳定性。试验工作器与活跃主容器在网络和挂载上隔离，并在迭代结束时拆除。循环前基线不需要再次运行代理：每个批量块已经包含在证据时由自动扫描捕获的原始记录，任务评估阶段直接对这些记录进行评分，以锁定

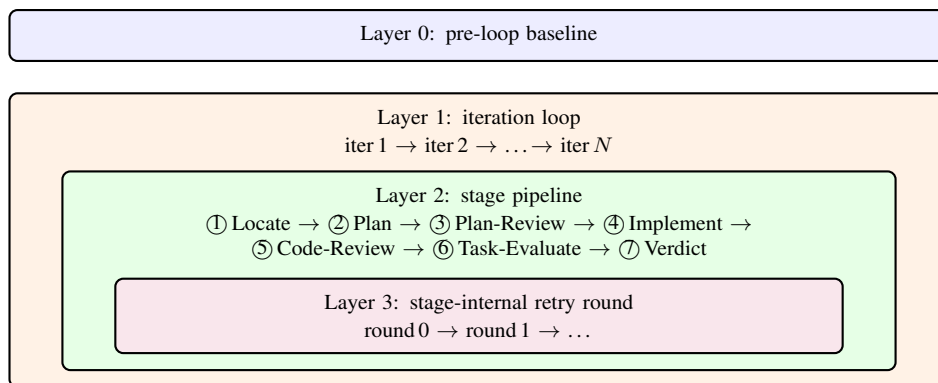


图 2: MOSS 演化的四个嵌套层级: 循环前基线 (第 0 层)、迭代循环 (第 1 层)、每次迭代的固定顺序七阶段流水线 (第 2 层), 以及围绕两个审查门的内部多轮循环 (第 3 层)。

循环其余部分的关键点集。这使得验证比基于单元测试或外部基准的门控更接近部署环境。

3.5 原地容器替换

标准部署模式不适合生产环境。代理是绑定到用户状态的单实例系统——会话、内存、凭据和代理配置都存储在用户状态卷中——因此蓝绿或金丝雀部署将需要粘性会话路由或非平凡的状态迁移。用户状态还必须在替换后保留, 以便用户体验到更好的代理而不是被替换的代理, 这是最小脚手架自演化代理所不面临的约束。回滚是强制性的, 因为试验工作器在隔离环境中针对批量任务运行, 没有活跃容器的用户状态或生产流量, 通过试验的候选者仍可能在活跃环境中退化。

收敛并不会自动提升候选版本。在收到“已收敛”判定后, 演化循环将该批次标记为待应用状态, 并触发一个演化收敛的网络钩子, 网关将其作为系统消息路由给智能体; 提升操作需要用户调用 `moos evo apply` 命令, 通常由智能体在对话中代为执行。应用请求通过 HTTP POST 发送至网关, 网关原子性地写入一个交换请求文件; 主机守护进程的交换监督器每两秒轮询该路径一次, 一旦检测到请求, 便使用候选镜像重启基座容器。随后进入 90 秒的探测窗口, 每 5 秒采样一次; 每次探测执行四项健康检查 (心跳新鲜度 ≤ 30 秒、容器运行状态以及两项基座层命令行状态探测), 连续三次通过则提交交换, 否则监督器执行回滚。回滚目标从独立的“最近已知良好”镜像记录中读取, 而非来自交换请求本身, 因此过期的请求不会使系统陷入回滚循环。无论结果如何, 监督器都会向已刷新的网关发送一个应用完成的网络钩子——携带成功或已回滚状态——智能体将其作为系统消息接收, 并重新适应新状态。用户状态卷原样挂载到新容器中, 使会话、记忆、凭据和智能体配置得以跨交换保留。

4 案例研究

本节通过一个案例研究展示 MOSS 演化循环, 使用一批受控的 `claweval` [叶等人, 2026年] 基准任务作为输入, 实证证明该循环能够产生有效的测试框架级修复。我们将四个合规审计任务作为失败批次输入循环, 端到端运行流水线 (循环前基线 → 阶段流水线 → 试验验证 → 宿主守护进程交换), 并在相同的四个任务上测量迭代 1 候选镜像与基线镜像的差异。分数差异以及智能体输出的定性变化, 共同证明该循环能够弥合从受控失败信号到部署智能体改进之间的差距。

4.1 实验设置：任务与基线

该批次包含四个爪评（claweval）任务，属于运营/合规审计类别。T141zh / T142（服务水平协议合规审计，中文/英文）要求智能体列出 P1 违规工单，并根据配置模拟服务提供的分层规则计算每个工单的服务水平协议合规性。T137zh / T138（补货链检查，中文/英文）要求智能体追踪多服务链——调度任务、集成配置、库存水平——并报告哪些补货任务失败、哪个集成损坏以及哪些库存短缺。每对任务在两种语言变体下共享同一任务。我们将这四个任务既作为输入批次用于进化，也作为测试集用于对进化后候选者重新评分——这是一种受控设置，使同一评分器能够见证循环的两端。

被演化的智能体是 OpenClaw，其底层模型为 DeepSeek V3.2 [刘等人，2025年]。基线运行在 claweval 评分器的 [0, 1] 量表上得分为 0.21 – 0.33，均值接近 0.25，远低于其 0.75 的通过阈值。基线试验记录显示两类任务族中反复出现的失败模式：在 SLA 合规任务对中，智能体通常只列出部分相关工单，将其余工单标记为“响应数据不完整，合规状态无法确定”，并且偶尔在相邻工单之间错误归属客户名称；在补货链任务对中，响应同样不完整，调度任务、集成和库存之间的链条断裂或缺失。这些正是演化循环被要求修复的失败模式。

爪评评分器仅作为读者的外部数值见证；它与 MOSS 内部的“任务-评估”阶段是分离的，后者生成驱动裁决的定性关键点矩阵（§ 3.3）。在此使用基准批次以可复现性换取真实性——在进化前后运行相同的四个任务提供了受控测量，而有机用户会话无法锚定这种测量。

4.2 MOSS 如何处理该批次

给定此输入批次，MOSS 运行循环前基线：“任务-评估”阶段对四个任务中每个任务的预捕获基线记录进行评分，生成基线关键点矩阵，该矩阵锁定循环其余部分的关键点集。该矩阵在工具排序、信息提取和结果报告方面表现薄弱或缺失。

定位阶段摄取基线转录，并识别出测试框架在多工具执行模式的工具结果处理中存在的覆盖缺口。演化中的智能体通常会选择测试框架的通用执行路径，而非中介器所围绕的语义工具，而中介器对该路径没有标注分支。当智能体将多个查询批量合并到单个外壳构造中时，测试框架的调度-综合管道中的次要解析问题加剧了该缺口，使下游消费者获得合并的、部分归属的输出。这两个表面共同解释了基线试验转录中可见的半答案现象。

规划阶段将诊断转化为测试框架内部的双表面修复：增加一个标注分支，在执行路径返回多实体负载时注入显式使用提示；以及一个调用前拒绝门，阻止特定的批量外壳模式，从而引导智能体转向独立的、可单独解析的调用。规划审查阶段无修改地批准了该设计。实现阶段将其作为内部 OpenClaw 仓库上的单个提交落地，修改了三个文件，新增 177 行、删除 1 行：在工具结果中介器中新增标注分支及支持辅助函数，在工具调用前钩子链中增加调用前检查，以及新增一个覆盖两个表面的中介器测试文件。该变更触及测试框架本身，而非任何文本可变工件，将修复定位在 § 1 论证先前系统无法触及的位置。

代码审查阶段批准，主机守护进程构建候选镜像，试验工作器针对该镜像重放批量任务。图 3 勾勒了由此产生的迭代-1 时间线。任务评估阶段对转录重新评分；裁决阶段将新的关键点矩阵与基线进行比较，观察到工具排序和结果报告关键点的广泛提升，候选方案收敛。

4.3 结果：迭代-1 结果

收敛后，主机守护进程执行 § 3.5 中描述的就地容器替换；对于本次基准运行，用户同意门被自动确认，以便评估管道能够完成

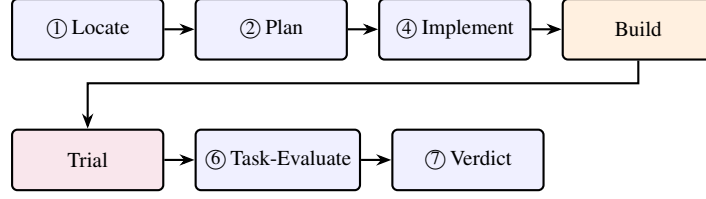


图 3: 迭代-1 迹，涵盖 MOSS 执行的所有阶段；为紧凑起见，省略了阶段 3（规划审查）和阶段 5（代码审查）门。

无需交互式审查。随后，收敛后的候选镜像针对相同的四个任务重新运行，评分器得分报告于表 2。

表 2: 各任务 ClawEval 评分器得分（每任务 3 次试验均值）在迭代 1 交换前后的对比。

Task	Baseline	Iter 1	Δ
T141zh_sla_compliance_audit	0.3273	0.5330	+0.2057
T142_sla_compliance_audit	0.2527	0.5453	+0.2926
T137zh_restock_chain_check	0.2213	0.4567	+0.2354
T138_restock_chain_check	0.2090	0.9049	+0.6959
mean	0.2526	0.6100	+0.3574

四项任务均值从 0.2526 升至 0.6100。最大增益出现在 T138，升至 0.9049，三次试验均超过 0.75 的通过阈值。SLA 任务对（T141zh / T142）从 0.25 – 0.33 区间升至约 0.53 – 0.55：标注路径弥补了最大缺陷，而每张工单的时间差计算和 SLA 层级分类仍存在自身难度，一次迭代未能解决。T137zh 增益最小，为 +0.2354，其中一次试验达到通过标准，两次低于阈值。

得分与智能体的实际输出相对应。此前在 § 4.1 中描述为半答案的 SLA 合规任务记录，现在返回每张工单的 SLA 层级分类及汇总摘要（“6 张 P1 工单中有 2 张违反 SLA，影响客户 B 和客户 D，平均超时 2.3 小时”）；补货链记录同样返回完整链条。模型和任务定义未变；测试框架的变更是直接原因。

5 相关工作

5.1 智能体系统

能力强大的对话模型 [Achiam et al., 2023] 将大语言模型重新定位为系统组件而非演示，由此引出了如何让其行动的问题。工具增强的语言模型首先给出了答案：Toolformer [Schick et al., 2023] 证明大语言模型能够学习何时调用应用程序编程接口，ToolLLM [Qin et al., 2024] 将该思想扩展到数千种真实工具，使模型从文本生成器转变为可调用的执行器。ReAct [Yao et al., 2022] 通过将思维链推理与环境动作交替进行，将执行器闭合为循环，由此产生的思考/行动周期已成为此后所有智能体的内核。当一个智能体可行后，在单一推理框架内将角色分配给多个智能体便成为自然之举——MetaGPT [Hong et al., 2024]、AutoGen [Wu et al., 2024]、ChatDev [Qian et al., 2024] 和 CAMEL [Li et al., 2023] 各自展示了不同形式的角色专业化——该设计空间现已在综述层面得到梳理 [Wang et al., 2024, Guo et al., 2024]。这一脉络现已延伸至应用级生产智能体：Anthropic 的 Claude 助手 [Anthropic, 2024] 和开源的 OpenClaw [OpenClaw, 2024] 作为完整的生产级智能体系统运行——多通道、多用户服务，具备持久状态和完整的测试框架层，负责路由、门控和生命周期管理。

5.2 自进化智能体

关于自进化智能体的研究沿两条路线展开。学术路线在最小支架上将源码级自进化确立为一种可行原语。SICA [罗贝恩斯等人, 2025] 表明智能体能够编辑自身实现并提升其 SWE-Bench 得分, 从而解决了可行性问题。达尔文哥德尔机 [张等人, 2025] 将同一原语重新表述为对智能体变体档案的开放式搜索; HyperAgents [张等人, 2026] 通过使元过程本身可编辑, 进一步推进了这种风格。Meta-Harness [李等人, 2026] 分离出更精细的信号: 将过去的执行迹暴露给编码智能体提议者, 能够比仅靠基准得分反馈更强地增强迭代改进。应用级路线将进化扩展到已部署的智能体, 但将其范围限制在文本可变工件上。Hermes Agent [努斯研究, 2024] 与编译并搜索提示的 DSPy [哈塔布等人, 2023] 和 GEPA [阿格拉瓦尔等人, 2025] 优化器相结合, 构成了这一侧最雄心勃勃的集群; Capability Evolver [王等人, 2026]、SkillClaw [马等人, 2026]、GenericAgent [梁等人, 2026] 和 EvoAgentX [王等人, 2025] 各自占据不同的文本可变层——行为基因与记忆胶囊、共享技能语料库、Markdown 标准操作程序、提示与流程图——且均未触及执行框架。MOSS 将两者结合: 将源码级进化应用于应用级基底, 纳入执行框架层, 并以部署失败信号取代基准得分。

6 结论

我们认为, 源级适配对于自进化智能体而言, 是一种比先前应用级系统所限定的文本可变范围更为根本的通用媒介: 它具备图灵完备性, 是任何文本可变设计空间的严格超集, 效果具有确定性, 并且在长上下文漂移下保持稳定。MOSS 在生产级智能体基座上实例化了这一论点, 将可编辑范围从技能文件、提示配置、记忆模式和工作流图扩展到智能体框架本身, 即路由、状态管理、钩子排序和调度所在之处。将进化锚定于自动整理的批量生产故障证据, 执行确定性的多阶段流水线, 将代码修改委托给可插拔的外部编码智能体命令行接口, 通过在临时试验工作节点中对候选镜像重放该批量证据来验证候选方案, 并通过用户同意门控的就地容器替换以及健康探针门控的回滚来推广候选方案, 共同闭合了从真实用户故障到已部署框架级修复的回路; 在 OpenClaw 上, 这一单一回路在无人工干预的情况下, 将四项任务的平均评分从 0.25 提升至 0.61。

参考文献

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. Anthropic technical report, March 2024. URL <https://www.anthropic.com/claude-3-model-card>. Model card for the Claude 3 family released March 2024; documents capabilities, benchmarks, and safety evaluations for Opus, Sonnet, and Haiku.
- Anthropic. Claude Code. <https://www.anthropic.com/claude-code>, 2025. Anthropic’s command-line coding agent for the Claude model family.
- Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- Hmbown. DeepSeek-TUI: Terminal UI for DeepSeek. <https://github.com/Hmbown/DeepSeek-TUI>, 2025. Terminal-UI coding agent for DeepSeek models.

- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.
- Jiaqing Liang, Jinyi Han, Weijia Li, Xinyi Wang, Zhoujia Zhang, Zishang Jiang, Ying Liao, Tingyun Li, Ying Huang, Hao Shen, et al. Genericagent: A token-efficient self-evolving llm agent via contextual information density maximization (v1. 0). *arXiv preprint arXiv:2604.17091*, 2026.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026.
- Nous Research. Hermes Agent: The self-improving AI agent built by Nous Research. GitHub repository, 2024. URL <https://github.com/NousResearch/hermes-agent>. Application-level self-evolving agent with a built-in learning loop: autonomous skill creation after complex tasks, skills self-improve during use, agent-curated memory, and procedural memory.
- Nous Research. Hermes Agent Self-Evolution: Evolutionary self-improvement for Hermes Agent. GitHub repository, 2026. URL <https://github.com/NousResearch/hermes-agent-self-evolution>. Companion project to Hermes Agent that evolves skills, tool descriptions, system prompts, and code via DSPy + GEPA; gates every variant on full pytest pass and size limits (skills $\leq$ 15KB, tool descriptions $\leq$ 500 chars).
- OpenAI. OpenAI Codex CLI. <https://github.com/openai/codex>, 2025. Open-source command-line coding agent backed by OpenAI models.
- OpenClaw. OpenClaw — Personal AI Assistant. GitHub repository, 2024. URL <https://github.com/openclaw/openclaw>. Open-source multi-channel agentic system; supports WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, Microsoft Teams, Matrix, Feishu, and other channels.
- OpenCode project. OpenCode. <https://opencode.ai>, 2025. Multi-provider open-source coding-agent CLI.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 15174–15186, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717, 2024.

- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Junjie Wang, Yiming Ren, and Haoyang Zhang. From procedural skills to strategy genes: Towards experience-driven test-time evolution. *arXiv preprint arXiv:2604.15097*, 2026.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. Evoagentx: An automated framework for evolving agentic workflows. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 643–655, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026.